

Analiza experimentală comparativă a algoritmilor de sortare

Influența structurii de date, distribuției inputului și implementării asupra performanței

Alexandru-Gabriel BUTOI

Student anul I, Universitatea de Vest din Timișoara

Facultatea de Informatică

Specializarea Informatică, Română

`alexandru.butoi06@e-uvt.ro`

2026

Rezumat

Lucrarea prezintă o analiză experimentală comparativă a mai multor algoritmi de sortare, testați pe structuri de date diferite: **array**, **vector** și listă simplu înlanțuită. Studiul urmărește diferența dintre complexitatea teoretică și performanța practică, cu accent pe localitatea în memorie, distribuția datelor de intrare, costurile auxiliare, paralelizare și calitatea implementării. Rezultatele confirmă diferența majoră dintre algoritmii de complexitate $O(n^2)$ și cei de complexitate $O(n \log n)$, dar arată și că algoritmi aflați în aceeași clasă teoretică pot avea timpi reali foarte diferiți. Lucrarea discută comportamentul Merge Sort pe listă față de **array**, diferența dintre Merge Sort paralel și QuickSort cu pivot random, limitele implementării manuale a IntroSort și importanța păstrării datelor experimentale pentru cercetări ulterioare.

Cuvinte-cheie: algoritmi de sortare, complexitate, QuickSort, MergeSort, IntroSort, structuri de date, localitate în memorie, analiză experimentală.

1. Introducere

Sortarea este una dintre problemele fundamentale ale informaticii și apare în foarte multe contexte practice: procesarea datelor, baze de date, sisteme de fișiere, algoritmi de căutare, aplicații numerice și programe care lucrează cu volume mari de informație. Deși sortarea este o temă clasică, comportamentul real al algoritmilor nu este întotdeauna la fel de simplu precum pare din analiza teoretică.

În mod obișnuit, algoritmii de sortare sunt comparați prin complexitatea lor asimptotică. De exemplu, Bubble Sort, Selection Sort și Insertion Sort sunt asociați cu o complexitate de ordin $O(n^2)$, iar Merge Sort, QuickSort și IntroSort sunt asociați cu ordinul $O(n \log n)$, cel puțin în cazurile uzuale sau medii. Totuși, într-un program real, timpul efectiv de execuție depinde și de alți factori: structura de date folosită, modul în care datele sunt așezate în memorie, costurile de copiere, accesul secvențial sau direct, distribuția inputului și detaliile concrete ale implementării. Manualele clasice de algoritmică explică foarte bine comportamentul teoretic al acestor metode.¹ Totuși, scopul acestei lucrări nu este doar reluarea teoriei, ci verificarea ei prin măsurători. În

¹Pentru prezentarea generală a algoritmilor de sortare și a analizei de complexitate au fost folosite lucrările lui Cormen et al. [5], Knuth [10] și Sedgewick și Wayne [16].

practică, doi algoritmi cu aceeași complexitate pot avea timpi foarte diferiți, iar aceeași metodă poate funcționa diferit pe `array`, `vector` sau listă simplu înlănțuită.

Problema cercetată poate fi formulată astfel: în ce măsură alegerea algoritmului de sortare, a structurii de date și a implementării influențează timpul real de execuție? Întrebarea este importantă deoarece, în aplicații reale, nu este suficient ca un algoritm să fie corect. El trebuie să fie și utilizabil pentru dimensiunea datelor pe care trebuie să le proceseze.

În această lucrare au fost analizați algoritmi elementari, precum Bubble Sort, Selection Sort și Binary Insertion Sort, dar și algoritmi mai eficienți, precum Merge Sort, QuickSort, Merge Sort paralel și IntroSort. Rezultatele au fost interpretate în raport cu structura de date folosită, cu distribuția inputului și cu diferențele dintre implementările manuale și cele standard.

2. Contribuția lucrării

Contribuția acestei lucrări nu constă în propunerea unui algoritm nou de sortare, ci în realizarea unei comparații experimentale între algoritmi cunoscuți, urmărind modul în care performanța lor se modifică în funcție de structura de date, distribuția inputului și implementarea concretă. Scopul principal este evidențierea diferenței dintre complexitatea teoretică și timpul real de execuție.

Lucrarea aduce trei contribuții principale. Prima constă în compararea algoritmilor pe mai multe structuri de date: `array`, `vector` și listă simplu înlănțuită. Această comparație este importantă deoarece arată că același algoritm poate avea timpi diferiți în funcție de localitatea în memorie și de modul de acces la elemente.

A doua contribuție constă în analiza comparativă a algoritmilor de complexitate $O(n^2)$ și $O(n \log n)$ pe volume diferite de date. Rezultatele arată că diferența dintre cele două clase nu este doar teoretică, ci devine rapid vizibilă în practică, prin trecerea de la timpi de ordinul secundelor la estimări de ordinul orelor, zilelor sau chiar anilor pentru algoritmii quadratici.

A treia contribuție constă în evidențierea influenței implementării. Rezultatele obținute pentru QuickSort, Merge Sort paralel și IntroSort manual arată că performanța finală nu depinde doar de ideea algoritmică, ci și de detalii precum alegerea pivotului, costurile de copiere, memoria auxiliară, pragurile folosite și overhead-ul recursiv sau paralel.

Prin urmare, lucrarea susține ideea că alegerea unui algoritm de sortare nu trebuie făcută doar pe baza clasei de complexitate, ci printr-o analiză care combină teoria cu măsurători experimentale.

3. Cadrul teoretic și bibliografic

Problema sortării poate fi formulată simplu: pentru un șir de valori $a_1, a_2, \dots, a_n$, se cere obținerea unei permutări a elementelor astfel încât:

$$a_1 \leq a_2 \leq \dots \leq a_n.$$

În această lucrare, ordonarea se face după ordinea numerică naturală, iar accentul nu cade pe demonstrarea corectitudinii algoritmilor, ci pe comportamentul lor practic.

Bubble Sort, Selection Sort și Insertion Sort sunt algoritmi elementari, importanți mai ales didactic. Ei sunt ușor de înțeles și de implementat, dar au cost quadratic în cazurile generale. Din acest motiv, devin rapid nepractici pentru inputuri mari. În schimb, Merge Sort și QuickSort fac parte din familia algoritmilor divide et impera și au, în mod obișnuit, performanțe mult mai bune pentru volume mari de date.

QuickSort este creditat prin lucrarea lui Hoare.² HeapSort este creditat prin lucrarea lui Williams.³ IntroSort este introdus de Musser ca algoritm hibrid introspectiv, care combină avantajele QuickSort cu garanțiile HeapSort.⁴ Pentru partea de implementări eficiente, lucrarea lui Bentley și McIlroy este relevantă deoarece arată că performanța unei funcții de sortare depinde foarte

²QuickSort a fost introdus de Hoare în articolul *Quicksort* [8].

³HeapSort este asociat cu lucrarea lui Williams [18].

⁴IntroSort este prezentat în lucrarea lui Musser [12].

mult de detalii de implementare, nu doar de algoritmul ales.⁵ Funcția `std::sort` din C++ este tratată ca reper practic, deoarece implementările standard sunt optimizate și respectă cerințe stricte de complexitate.⁶

Algoritm / con- cept	Sursă	Rol în lucrare
Bubble, Selection, Insertion	[5, 10, 16]	Fundamentare generală și didactică.
Merge Sort	[10, 5]	Analiza metodei divide et impera și a interclasării.
QuickSort	[8]	Creditarea algoritmului original și analiza partiționării.
HeapSort	[18]	Componentă relevantă pentru algoritmii hibridi.
IntroSort	[12]	Algoritm hibrid introspectiv.
Implementări eficiente	[3, 6]	Reper pentru optimizări practice și biblioteci standard.
Rezultate proprii	codul sursă și rezultatele experimentale	Baza pentru graficele și interpretările din lucrare.

Tabela 1: Rolul surselor utilizate în lucrare.

Din punct de vedere teoretic, algoritmii pot fi comparați după complexitate temporală, complexitate spațială, stabilitate, caracter in-place și sensibilitate la forma inputului. Totuși, această clasificare nu epuizează problema. În experimente, diferențele dintre structuri de date și dintre implementări au fost suficient de mari încât să modifice semnificativ interpretarea rezultatelor.

Algoritm	Complexitate medie	Memorie auxiliară	Stabilitate	Observație practică
Bubble Sort	$O(n^2)$	$O(1)$	stabil	simplic, dar slab pe date mari
Selection Sort	$O(n^2)$	$O(1)$	de regulă instabil	puține schimburi, multe comparații
Binary Insertion Sort	$O(n^2)$	$O(1)$	stabil	reduce comparațiile, dar nu deplasările
Merge Sort	$O(n \log n)$	$O(n)$	stabil	predictibil, dar folosește memorie auxiliară
QuickSort	$O(n \log n)$ mediu	$O(\log n)$	instabil	rapid în practică, in-place
IntroSort	$O(n \log n)$	$O(\log n)$	instabil	hibrid, dependent de implementare

Tabela 2: Caracteristici teoretice generale ale algoritmilor analizați.

4. Metodologia experimentală

Metodologia utilizată este experimentală și comparativă. Fiecare algoritm a fost rulat pe aceleași seturi de date, iar acolo unde implementarea a permis, rezultatele au fost colectate separat pentru `array`, `vector` și listă simplu înlanțuită. Această separare permite observarea influenței algoritmului, dar și a influenței structurii de stocare asupra timpului de execuție.

Testele au fost rulate pe următoarea configurație hardware și software:

- sistem de operare: Windows 11 Pro;
- mediu de execuție: Windows Subsystem for Linux, utilizat prin Visual Studio Code;

⁵Bentley și McIlroy discută problema ingineriei unei funcții de sortare performante în [3].

⁶Pentru comportamentul funcției `std::sort` a fost consultată documentația cplusplus [6].

- procesor: Intel Core i7-13650HX;
- memorie RAM: 16 GB, 4800 MT/s;
- placă video: NVIDIA GeForce RTX 4060 Laptop GPU.

Placa video este menționată doar ca parte a configurației generale, deoarece algoritmi analizați au fost executați pe procesor. Nu au fost aplicate optimizări manuale ale sistemului, precum overclocking, limitarea explicită a frecvenței sau controlul manual al afinității thread-urilor.

Seturile de date au inclus mai multe dimensiuni, de la cazuri foarte mici până la volume foarte mari. Au fost folosite distribuții random, date sortate, date invers sortate, date egale, date aproape sortate, date cu multe duplicate și date din intervale numerice restrânse. Această diversitate este necesară deoarece distribuția inputului poate influența numărul de comparații, numărul de mutări, echilibrul partiționării și comportamentul memoriei cache.

Timpul raportat reprezintă timpul efectiv al sortării. Citirea datelor a fost realizată înainte de pornirea cronometrării, iar scrierea rezultatului după oprirea acesteia. Astfel, măsurătoarea urmărește comportamentul algoritmului de sortare, nu performanța sistemului de fișiere. Această separare este importantă deoarece timpul de I/O poate masca diferențele reale dintre algoritmi. Pentru fiecare test s-au urmărit următoarele elemente: algoritmul utilizat, structura de date, dimensiunea inputului, tipul datelor, distribuția inputului și timpul de execuție. În cazul algoritmilor paraleli, interpretarea rezultatelor a avut în vedere și costurile suplimentare introduse de crearea și sincronizarea thread-urilor.

Modelul de cost folosit în interpretare nu se limitează la numărul de comparații. Au fost luate în considerare și mutările de elemente, copierea datelor, memoria auxiliară, accesul la memorie, localitatea cache, apelurile recursive și overhead-ul paralelizării. Din acest motiv, rezultatele sunt interpretate ca efect al interacțiunii dintre algoritm, structură de date și implementare.

Codul sursă și datele asociate experimentelor sunt disponibile public în repository-ul lucrării.⁷ Pentru extinderea reproductibilității, studiul poate fi completat ulterior cu versiunea exactă a compilatorului, flagurile de optimizare, seed-urile folosite pentru generarea datelor random și numărul exact de repetări pentru fiecare test.

5. Protocolul experimental

Pentru a menține comparația corectă, algoritmi au fost testați pe aceleași seturi de date. Fiecare rundă a urmat aceeași ordine logică: încărcarea sau generarea datelor, construirea structurii de date corespunzătoare, pornirea cronometrării, executarea sortării, oprirea cronometrării și verificarea rezultatului.

Această organizare reduce riscul ca diferențele observate să provină din pași exteriori sortării. De exemplu, dacă timpul de citire ar fi fost inclus în măsurătoare, rezultatele ar fi putut fi influențate de performanța discului sau de sistemul de fișiere. Prin separarea timpului de sortare de timpul de I/O, comparația rămâne mai apropiată de comportamentul real al algoritmilor.

Dimensiunile inputului au fost alese astfel încât să evidențieze atât comportamentul pe cazuri mici, cât și scalarea pe cazuri mari. Pentru algoritmi $O(n^2)$, unele rulări foarte mari devin impracticabile, motiv pentru care interpretarea include și estimări sau opriri ale testelor care ar fi consumat timp excesiv. Această alegere este justificată deoarece scopul nu este rularea inutilă a unui test de ordinul zilelor, ci observarea creșterii timpului și compararea ordinelor de mărime.

Pentru algoritmi $O(n \log n)$, testele mari sunt mult mai relevante, deoarece aceștia rămân executabili pentru volume unde algoritmi quadratici nu mai sunt practici. În această categorie, comparația urmărește nu doar clasa de complexitate, ci și costurile ascunse: memoria auxiliară la Merge Sort, partiționarea la QuickSort, pragurile la IntroSort și overhead-ul de sincronizare în cazul Merge Sort paralel.

Rezultatele au fost sintetizate în grafice logaritmice. Alegerea scării logaritmice este potrivită deoarece diferențele dintre algoritmi devin foarte mari pe măsură ce dimensiunea inputului crește.

⁷https://github.com/alimovul/ScriereAcademica_Sortari

Într-o scară liniară, unele valori mici ar deveni greu de interpretat, iar comparația dintre clasele de complexitate ar fi mai puțin clară.

6. Algoritmii și structurile analizate

Bubble Sort, Selection Sort și Binary Insertion Sort au fost folosiți ca reprezentanți ai algoritmilor de complexitate $O(n^2)$. Bubble Sort compară elemente vecine și mută treptat valorile mari spre finalul șirului. Selection Sort caută repetat minimumul din partea nesortată și îl așază pe poziția corectă. Binary Insertion Sort folosește căutare binară pentru a găsi poziția de inserare, dar deplasarea elementelor rămâne costisitoare. De aceea, chiar dacă reduce numărul de comparații, nu elimină costul quadratic.

Merge Sort și QuickSort aparțin familiei divide et impera. Merge Sort împarte șirul în două părți, sortează recursiv fiecare parte și apoi interclasează rezultatele. Este un algoritm predictibil, dar are cost de memorie auxiliară și operații suplimentare de copiere. QuickSort alege un pivot și rearanjează elementele astfel încât valorile mai mici să fie într-o parte, iar cele mai mari în cealaltă. În implementarea testată, pivotul ales random a redus probabilitatea unor partiționări nefavorabile.

Merge Sort paralel a fost analizat pentru a observa dacă împărțirea lucrului pe mai multe fire de execuție aduce un avantaj practic clar. În mod teoretic, paralelizarea ar trebui să ajute pentru volume mari de date. În practică, însă, beneficiul depinde de costul sincronizării, de traficul în memorie și de modul în care sarcinile sunt împărțite.

IntroSort a fost analizat ca algoritm hibrid. Ideea de bază este puternică: se pornește cu QuickSort, iar dacă adâncimea recursiei devine prea mare, se trece la HeapSort pentru a evita degradarea. În implementările performante se folosesc și praguri pentru trecerea la Insertion Sort pe subsecvențe mici. În lucrarea de față, varianta manuală de IntroSort este tratată separat de `std::sort`, deoarece o implementare proprie nu are același nivel de rafinare ca o bibliotecă standard.

Structurile de date analizate au fost:

- **array**, cu stocare contiguă și acces direct;
- **vector**, tot cu stocare contiguă, dar cu mecanisme suplimentare oferite de biblioteca standard;
- listă simplu înlănțuită, cu noduri legate prin pointeri și acces secvențial.

Această alegere permite observarea diferenței dintre date stocate compact și date dispersate în memorie. **Array**-ul și **vector**-ul permit procesorului să folosească mai eficient cache-ul. Lista simplu înlănțuită poate fi convenabilă conceptual pentru unele operații, dar este penalizată prin dereferențieri repetate și localitate slabă.

7. Rezultate experimentale și interpretare

7.1. Algoritmii de complexitate $O(n^2)$

Rezultatele pentru Bubble Sort, Selection Sort și Binary Insertion Sort confirmă degradarea rapidă a performanței odată cu mărirea lui n . Acești algoritmi sunt ușor de înțeles și pot fi utili pentru dimensiuni mici, dar devin foarte repede nepotriviti pentru volume mari.

Bubble Sort a ajuns, la $n = 100000$, la zeci de secunde pe structurile testate. Pentru dimensiuni mai mari, rularea nu mai este practică. Selection Sort are același tip de comportament, iar estimările pentru valori foarte mari ajung la zile sau chiar ani. Binary Insertion Sort este mai bun decât celelalte două în anumite cazuri, deoarece reduce numărul de comparații, dar nu schimbă problema fundamentală: elementele trebuie deplasate, iar costul rămâne $O(n^2)$.

Un exemplu clar apare la Binary Insertion Sort. Pentru $n = 10^6$, timpul ajunge la aproximativ 310.51 secunde pe **vector**, 125.14 secunde pe **array** și 210.10 secunde pe listă. Pentru $n = 10^8$, estimările urcă la ordinul zilelor. Acest lucru arată că o îmbunătățire locală a metodei nu este suficientă atunci când ordinul de complexitate rămâne quadratic.

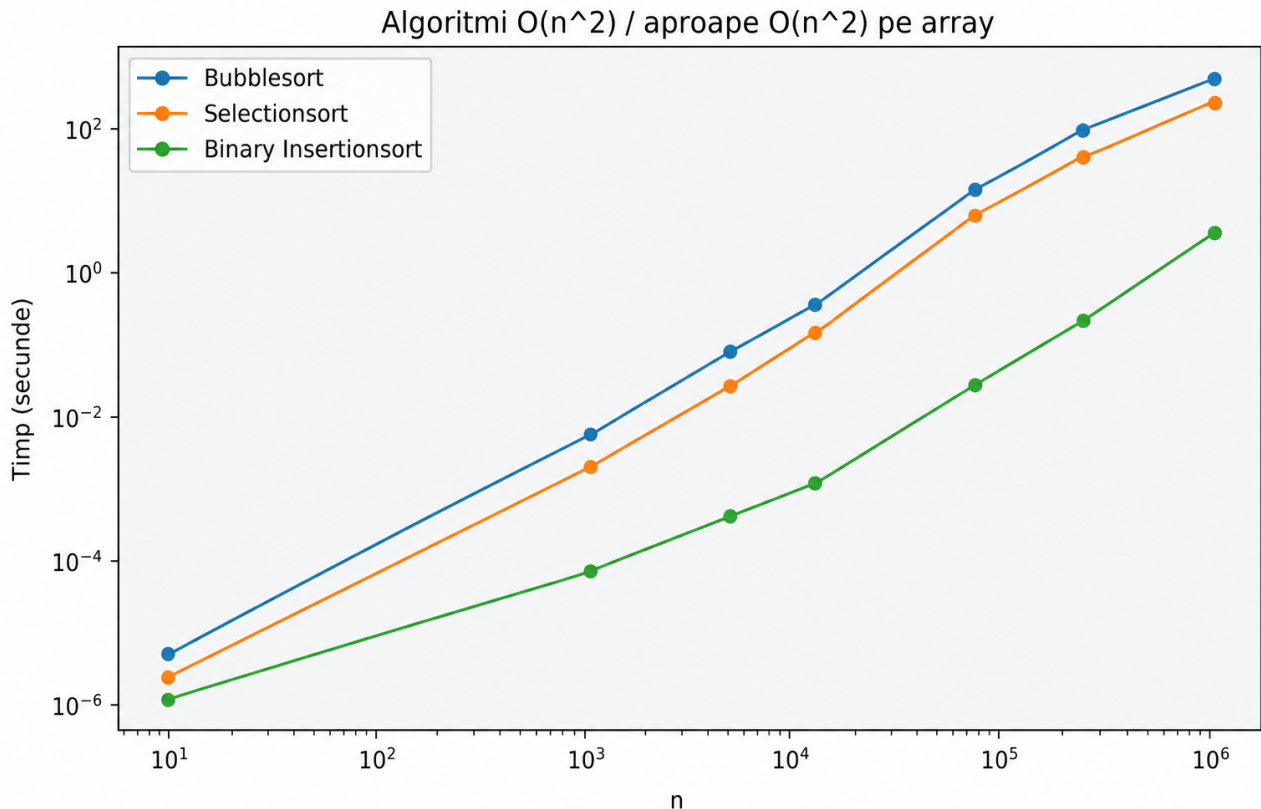


Figura 1: Evoluția timpului pentru algoritmi de complexitate $O(n^2)$, în scară logaritmică.

Graficul evidențiază creșterea rapidă a timpului. Diferențele dintre Bubble Sort, Selection Sort și Binary Insertion Sort pot conta pe valori mici sau medii, dar pentru dimensiuni mari toate ajung în zona impracticabilă. Concluzia este că algoritmi $O(n^2)$ trebuie evitați pentru seturi mari de date, indiferent de structura folosită.

7.2. Algoritmi de complexitate $O(n \log n)$

QuickSort, Merge Sort și IntroSort au avut un comportament mult mai bun. Pentru dimensiuni la care algoritmi $O(n^2)$ devin inutilizabili, acești algoritmi rămân în zona secundelor sau zecilor de secunde. Totuși, rezultatele arată că aceeași clasă de complexitate nu garantează același timp real.

QuickSort a fost cel mai eficient în testele mari dintre implementările analizate. Pentru $n = 10^8$, timpul a fost aproximativ 4.10 secunde pe **vector**, 3.85 secunde pe **array** și 35.00 secunde pe listă. Pentru $n = 5 \cdot 10^8$, timpul a fost aproximativ 23.50 secunde pe **vector**, 22.10 secunde pe **array** și 195.00 secunde pe listă. Diferența foarte mare dintre structurile contigue și listă arată încă o dată importanța localității în memorie.

Merge Sort a fost mai lent decât QuickSort pe volume mari. Pentru $n = 10^8$, timpul a fost aproximativ 47.52 secunde pe **vector**, 17.29 secunde pe **array** și 51.40 secunde pe listă. Pentru $n = 5 \cdot 10^8$, valorile au fost aproximativ 245.10 secunde pe **vector**, 88.51 secunde pe **array** și 289.41 secunde pe listă. Aceste valori arată că Merge Sort este stabil și predictibil ca idee, dar costurile de copiere și memoria auxiliară îl pot penaliza.

IntroSort manual a avut rezultate bune, dar nu la nivelul QuickSort-ului testat. Pentru $n = 10^8$, timpul a fost aproximativ 26.76 secunde pe **vector** și 14.17 secunde pe **array**. Pentru $n = 5 \cdot 10^8$, valorile au fost aproximativ 142.71 secunde pe **vector** și 71.76 secunde pe **array**. Aceste rezultate nu indică o problemă cu ideea de IntroSort, ci mai degrabă cu nivelul de optimizare al implementării manuale.

7.3. Influența structurii de date

Un rezultat clar al experimentelor este avantajul structurilor contigue. **Array**-ul a fost, în general, cea mai eficientă structură, mai ales pe dimensiuni mari. Explicația principală este localitatea

spațială: elementele consecutive sunt apropiate în memorie, iar procesorul poate încărca eficient blocuri continue de date.

Vector-ul are tot stocare contiguă și a avut rezultate apropiate de **array**, dar uneori a fost ușor mai lent. Diferența poate veni din modul de folosire a structurii, din verificări, din stilul implementării sau din detalii ale compilatorului. Lista simplu înlănțuită a fost penalizată cel mai mult la dimensiuni mari, mai ales pentru algoritmi care au nevoie de acces repetat la elemente.

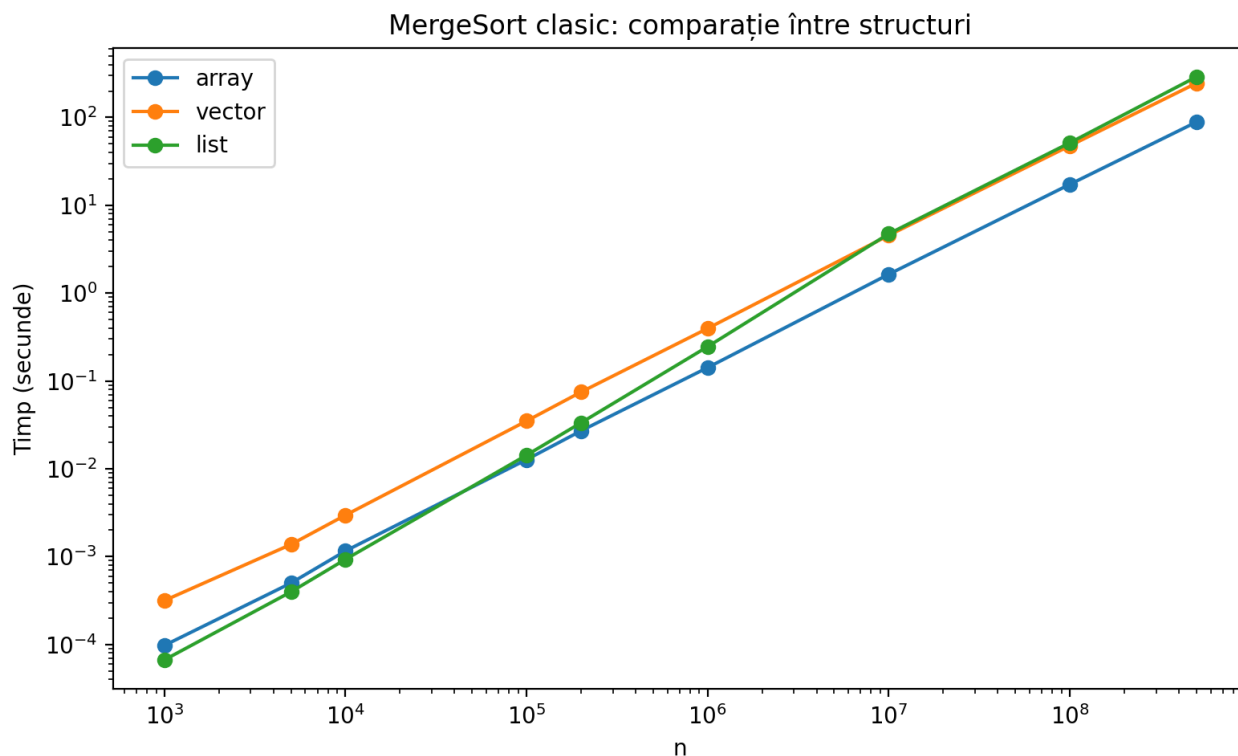


Figura 2: Merge Sort clasic: comparație între **array**, **vector** și listă simplu înlănțuită.

Rezultatul pentru Merge Sort este interesant. În teorie, Merge Sort se potrivește destul de bine cu listele, deoarece interclasarea se poate face prin relink-area nodurilor. Totuși, în experimente, lista a pierdut pe volume mari. Pentru $n = 10^7$, Merge Sort pe **array** a avut aproximativ 1.64 secunde, iar pe listă aproximativ 4.73 secunde. Pentru $n = 10^8$, **array**-ul a avut aproximativ 17.29 secunde, iar lista aproximativ 51.40 secunde.

Explicația este legată de costurile reale ale memoriei. Lista simplu înlănțuită presupune acces pointer-cu-pointer. Nodurile pot fi dispersate în memorie, ceea ce produce cache miss-uri și dereferențieri repetate. În schimb, **array**-ul oferă acces compact și predictibil. Astfel, chiar dacă la nivel conceptual lista pare potrivită pentru Merge Sort, la nivel practic structura contiguă rămâne mai eficientă.

7.4. Merge Sort paralel și QuickSort cu pivot random

Merge Sort paralel a redus timpul față de Merge Sort clasic, dar nu a reușit să depășească QuickSort-ul cu pivot random în testele mari. Pentru $n = 10^8$, Merge Sort paralel a avut aproximativ 14.00 secunde pe **vector** și 5.40 secunde pe **array**, în timp ce QuickSort a avut aproximativ 4.10 secunde pe **vector** și 3.85 secunde pe **array**. Pentru $n = 5 \cdot 10^8$, Merge Sort paralel a ajuns la aproximativ 70.03 secunde pe **vector** și 27.01 secunde pe **array**, iar QuickSort la aproximativ 23.50 secunde pe **vector** și 22.10 secunde pe **array**.

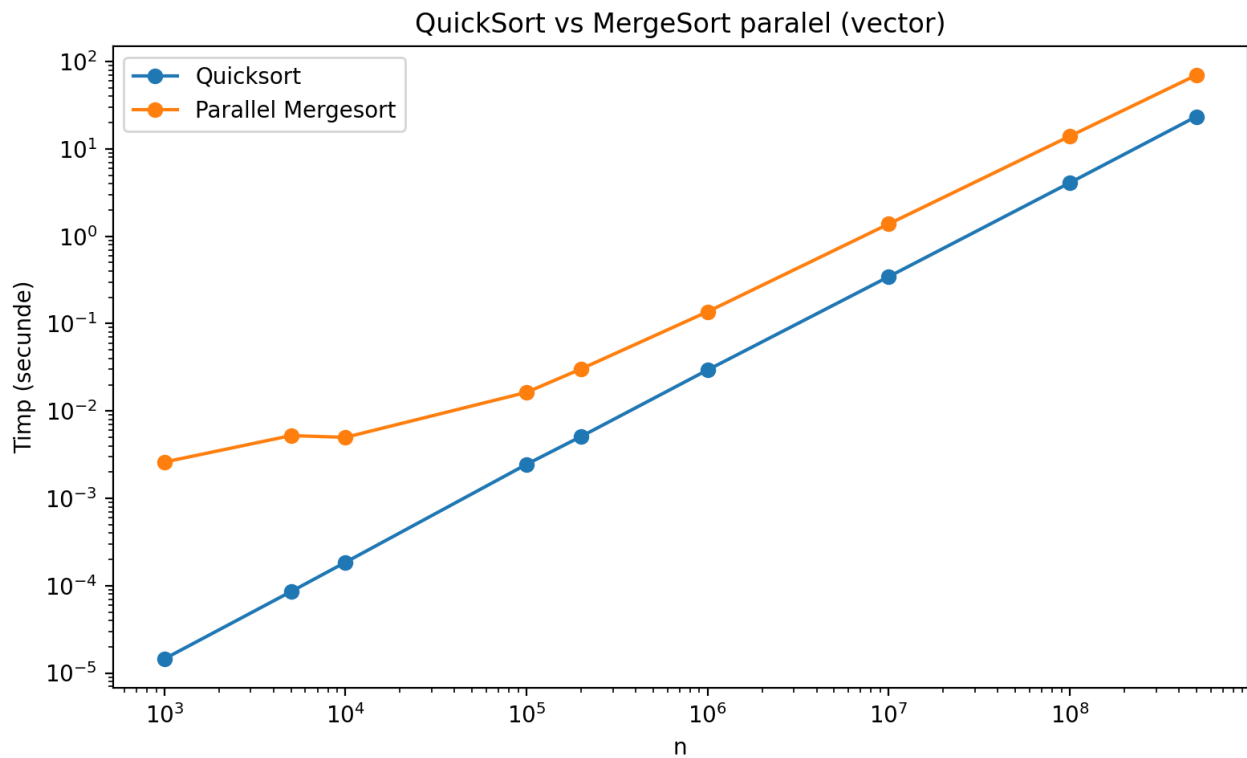


Figura 3: Comparație între QuickSort și Merge Sort paralel pe vector.

Această comparație arată că paralelizarea nu garantează automat un timp mai bun. Merge Sort paralel are costuri de creare și sincronizare a thread-urilor, plus costuri de interclasare și copiere. Chiar dacă lucrul este împărțit pe mai multe fire de execuție, traficul suplimentar în memorie poate limita câștigul.

QuickSort are avantajul că lucrează in-place și produce mai puțin trafic auxiliar. Alegerea random a pivotului reduce riscul unor partiționări foarte dezechilibrate, astfel încât în testele realizate algoritmul a rămas foarte competitiv. Rezultatul nu invalidează ideea de paralelizare, dar arată că aceasta trebuie evaluată experimental, nu presupusă ca superioară.

7.5. IntroSort manual și importanța implementării

IntroSort este, teoretic, un algoritm foarte solid deoarece combină QuickSort cu HeapSort și, în implementările performante, cu Insertion Sort pentru subsecvențe mici. Totuși, implementarea manuală testată a fost mai lentă decât ceea ce se așteaptă de la o implementare standard optimizată.

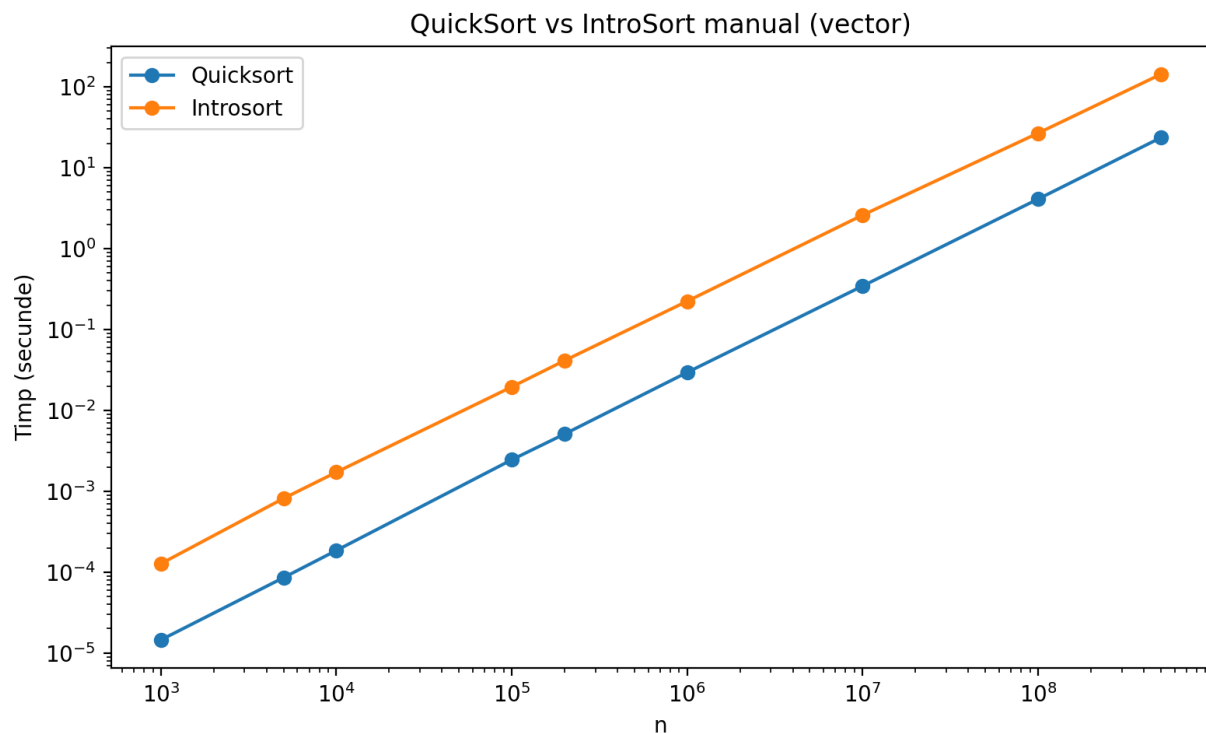


Figura 4: QuickSort vs IntroSort manual pe vector.

Diferența apare din detalii concrete: alegerea pragurilor, strategia de partiționare, reducerea recursiei, modul de trecere la HeapSort și optimizările compilatorului. O implementare STL este rezultatul unei rafinări serioase, nu doar o transcriere a algoritmului din manual. De aceea, concluzia corectă nu este că IntroSort este slab, ci că implementarea manuală trebuie analizată separat de implementările standard.

Această observație este importantă pentru întreaga lucrare. Performanța unui algoritm nu depinde doar de complexitatea sa, ci și de modul concret în care este scris codul. Două implementări ale aceleiași idei pot avea rezultate diferite, mai ales la dimensiuni mari.

7.6. Distribuția inputului

Distribuția datelor de intrare influențează semnificativ timpul de execuție. Datele random, sortate, invers sortate, aproape sortate, egale sau cu multe duplicate pot produce comportamente diferite. Diferențele apar din numărul de comparații, numărul de mutări, echilibrul partiționării și modul în care algoritmul profită sau nu de ordinea existentă.

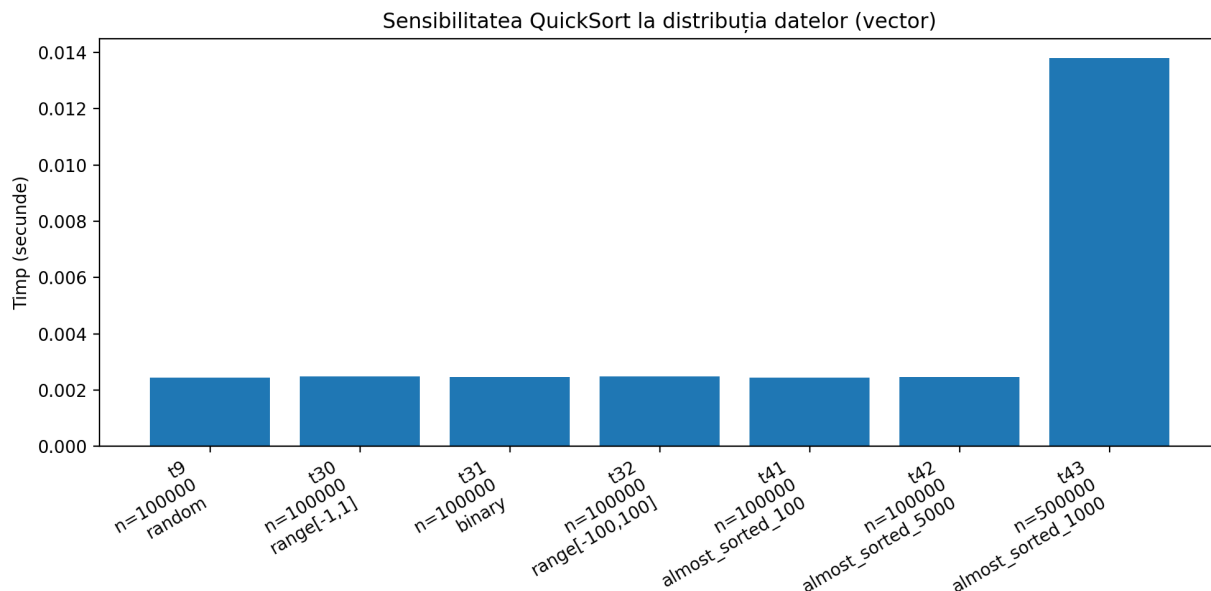


Figura 5: Sensibilitatea QuickSort la distribuția datelor pentru teste reprezentative.

Testarea doar pe date random nu este suficientă. Datele aproape sortate pot favoriza anumite metode, în timp ce datele invers sortate sau cu multe duplicate pot pune presiune pe altele. În cazul QuickSort, distribuția inputului este foarte importantă deoarece influențează direct partiționarea. Alegerea random a pivotului a funcționat bine în testele realizate, dar rămâne relevantă analiza unor cazuri special construite pentru a observa limitele implementării.

7.7. Sinteza rezultatelor

Rezultatele pot fi sintetizate prin câteva idei. Prima este că diferența dintre $O(n^2)$ și $O(n \log n)$ nu este doar teoretică, ci devine foarte vizibilă în practică. A doua este că structura de date poate schimba semnificativ timpul de execuție, chiar dacă algoritmul rămâne același. A treia este că implementarea concretă contează aproape la fel de mult ca alegerea algoritmului.

Factor analizat	Efect observat	Consecință practică
Complexitatea algoritmului	separă metodele lente de cele scalabile	algoritmii $O(n^2)$ trebuie evitați pe volume mari
Structura de date	influențează localitatea în memorie	datele contigue sunt preferabile pentru performanță
Distribuția inputului	modifică numărul de comparații și partiționări	testarea trebuie făcută pe cazuri diverse
Implementarea concretă	poate schimba mult timpii	bibliotecile standard sunt reperi practice
Paralelizarea	introduce overhead suplimentar	trebuie justificată prin măsurători

Tabela 3: Sinteza factorilor care influențează performanța practică.

8. Comparatie cu literatura de specialitate

Rezultatele obținute sunt în acord cu literatura clasică privind algoritmii de sortare. Manualele de algoritmică arată că metode precum Bubble Sort, Selection Sort și Insertion Sort au cost quadratic în cazurile generale, ceea ce limitează utilizarea lor la inputuri mici [5, 10, 16]. Experimentele realizate confirmă această limitare: pentru dimensiuni mari, timpul de execuție crește rapid și devine nep practic.

În cazul QuickSort, rezultatele sunt compatibile cu ideea introdusă de Hoare [8]. Algoritmul poate fi foarte eficient în practică atunci când partiționarea este suficient de echilibrată, iar implementarea are costuri reduse. În testele realizate, QuickSort cu pivot random a fost una dintre cele mai rapide variante, mai ales pe structuri contigue precum `array` și `vector`.

În cazul Merge Sort, literatura evidențiază predictibilitatea algoritmului, dar și costul memoriei auxiliare [10, 5]. Rezultatele proprii confirmă această observație: Merge Sort are un comportament stabil, dar poate fi penalizat de copieri, interclasare și trafic suplimentar în memorie. Acest lucru devine vizibil mai ales în comparația cu QuickSort, care lucrează in-place și are constante practice mai bune în implementarea testată.

Un punct important al lucrării este influența structurii de date asupra performanței. Rezultatele obținute pentru `array`, `vector` și listă simplu înlanțuită sunt în acord cu studiile care arată importanța memoriei cache în sortare. LaMarca și Ladner [11] au arătat că performanța algoritmilor de sortare este influențată puternic de configurația cache-ului și de accesările în memoria principală. În același sens, rezultatele acestei lucrări arată că lista simplu înlanțuită este penalizată la volume mari din cauza accesului indirect și a localității slabe în memorie.

Această observație este completată și de studiile care tratează simultan cache-ul și branch prediction-ul [4]. În lucrarea de față, analiza s-a concentrat mai ales pe timpul total de execuție și pe structura de date, dar rezultatele sugerează că accesul la memorie nu poate fi ignorat. Chiar dacă doi algoritmi au aceeași complexitate asimptotică, modul în care aceștia accesează memoria poate modifica semnificativ timpul real.

O dimensiune care merită menționată în raport cu literatura modernă este branch prediction-ul. Kaligosi și Sanders [9] arată că predicția ramificațiilor poate influența serios performanța QuickSort. Astfel, alegerea pivotului nu afectează doar echilibrul partițiilor, ci și comportamentul procesorului la nivel de ramificații. Edelkamp și Weiss [7] merg mai departe și propun BlockQuicksort, o variantă proiectată tocmai pentru reducerea efectului negativ al branch misprediction-urilor. Aceste lucrări oferă o explicație suplimentară pentru diferențele dintre implementările simple și cele foarte optimizate.

În cazul IntroSort, rezultatele trebuie interpretate cu atenție. Musser [12] propune un algoritm hibrid care combină avantajele QuickSort cu garanțiile HeapSort. Totuși, implementarea manuală testată nu trebuie confundată cu implementările standard foarte optimizate. Bentley și McIlroy [3] arată că o funcție de sortare performantă nu este doar rezultatul alegerii unui algoritm bun, ci și al unei inginerii atente a pragurilor, cazurilor particulare și buclelor interne. Acest lucru explică de ce `std::sort` trebuie tratat ca etalon practic, nu ca simplă referință teoretică [6].

Rezultatele privind Merge Sort paralel se aliniază și cu literatura despre sortarea paralelă. Deși paralelizarea poate reduce timpul de execuție, aceasta introduce costuri de sincronizare, creare de thread-uri și trafic suplimentar în memorie. Tsigas și Zhang [17] discută performanța implementărilor paralele de QuickSort, iar Axtmann et al. [2] arată că sortarea paralelă modernă necesită algoritmi atent proiectați pentru arhitecturi multi-core. Prin comparație, rezultatele acestei lucrări arată că o paralelizare directă a Merge Sort nu garantează automat depășirea unui QuickSort eficient.

În literatura modernă există și algoritmi hibridi adaptați pentru date reale. Timsort, propus de Tim Peters [14], este proiectat pentru a valorifica secvențele deja ordonate din datele reale. Analizele asupra strategiilor de interclasare din Timsort [1] arată că datele aproape sortate pot schimba semnificativ alegerea algoritmului potrivit. De asemenea, Pattern-defeating Quicksort [13] urmărește combinarea cazului mediu rapid al QuickSort cu protecții împotriva unor tipare nefavorabile. Aceste direcții sunt relevante pentru rezultatele lucrării, deoarece testele au inclus și distribuții speciale, nu doar date random.

În final, lucrarea se aliniază cu literatura de specialitate prin confirmarea diferenței dintre algoritmi $O(n^2)$ și cei $O(n \log n)$, dar adaugă o perspectivă practică asupra structurii de date, localității în memorie și calității implementării. Rezultatele nu contrazic teoria, ci arată că teoria trebuie completată prin măsurători experimentale pentru a înțelege comportamentul real al algoritmilor.

9. Limite ale cercetării

Rezultatele obținute trebuie interpretate în limitele cadrului experimental folosit. Lucrarea oferă o comparație practică între mai mulți algoritmi de sortare, dar nu pretinde că valorile numerice

obținute sunt universal valabile. Timpul real de execuție depinde de procesor, memorie, compilator, sistem de operare, flaguri de optimizare și încărcarea sistemului în momentul rulării.

O primă limită este faptul că toate testele au fost realizate pe o singură configurație hardware și software. Această alegere asigură comparabilitatea internă a rezultatelor, dar nu garantează că aceleași valori absolute vor fi obținute pe alte sisteme. Cu toate acestea, comparațiile relative rămân relevante, deoarece algoritmii au fost testați în aceleași condiții.

O a doua limită ține de faptul că experimentele au fost realizate exclusiv pe procesor. Deși sistemul de testare include o placă video dedicată, aceasta nu a fost folosită pentru sortare. Prin urmare, lucrarea analizează comportamentul algoritmilor pe CPU, nu și pe arhitecturi GPU. Pentru volume foarte mari de date, sortarea pe GPU poate deveni relevantă, însă presupune algoritmi și modele de memorie diferite față de cele analizate aici [15].

O altă limită este legată de implementările manuale. Algoritmi precum QuickSort, Merge Sort paralel sau IntroSort pot avea performanțe diferite în funcție de detaliile concrete ale codului. Alegerea pivotului, pragurile folosite în algoritmii hibridi, modul de gestionare a recursiei și strategia de sincronizare a thread-urilor pot modifica semnificativ timpul final. Din acest motiv, rezultatele trebuie interpretate ca evaluare a implementărilor testate, nu ca verdict absolut asupra tuturor variantelor posibile.

În cazul IntroSort, această limită este deosebit de importantă. Implementarea manuală nu trebuie confundată cu `std::sort`, chiar dacă ambele se bazează pe idei introspective. Implementările standard sunt rafinate prin praguri calibrate, partiționare eficientă, optimizări ale compilatorului și tratarea cazurilor speciale. De aceea, diferența dintre IntroSort manual și `std::sort` nu indică o slăbiciune a algoritmului în sine, ci arată importanța calității implementării [12, 3, 6].

O limită suplimentară este că lucrarea măsoară timpul total de sortare, dar nu măsoară direct evenimente hardware precum cache miss-uri, branch misprediction-uri sau trafic efectiv în memoria principală. Aceste fenomene sunt discutate interpretativ, deoarece explică diferențe observate între `array`, `vector` și listă simplu înlănțuită, dar nu au fost măsurate cu instrumente dedicate. O analiză mai avansată ar putea folosi contoare hardware pentru a separa mai clar costul comparațiilor de costul accesului la memorie și al predicției ramificațiilor [11, 9, 7].

De asemenea, seturile de date folosite acoperă mai multe distribuții, dar nu epuizează toate cazurile posibile. Au fost analizate date random, sortate, invers sortate, aproape sortate, egale sau cu duplicate, însă există și inputuri construite special pentru a produce comportamente nefavorabile anumitor algoritmi. În special pentru QuickSort, testele anti-QuickSort sau datele cu tipare speciale pot evidenția limite care nu apar în rulările obișnuite.

O altă limită este că unele rezultate pentru volume foarte mari sunt tratate prin estimări sau prin oprirea rulărilor care ar dura prea mult. Această abordare este necesară practic, deoarece algoritmii de complexitate $O(n^2)$ pot ajunge la timpi de ordinul orelor, zilelor sau anilor. Totuși, estimările nu au aceeași valoare ca o rulare completă și trebuie interpretate ca indicatori ai ordinului de mărime, nu ca măsurători exacte.

În final, rezultatele ar fi mai solide dacă fiecare test ar fi repetat de mai multe ori, iar valorile finale ar fi exprimate prin medie, mediană, minim, maxim și abatere standard. O singură rulare poate fi influențată de procese de sistem, alocări de memorie sau variații temporare ale frecvenței procesorului. Chiar și așa, diferențele mari observate între clasele de algoritmi și între structurile de date sunt suficient de clare pentru a susține concluziile principale ale lucrării.

10. Direcții viitoare de cercetare

Rezultatele obținute deschid mai multe direcții de cercetare care pornesc direct din observațiile experimentale. Aceste direcții nu schimbă concluziile lucrării, ci le pot rafina printr-o analiză mai apropiată de comportamentul real al procesorului, al memoriei și al implementărilor moderne.

O primă direcție este analiza mai detaliată a motivului pentru care Merge Sort pe listă simplu înlănțuită pierde față de Merge Sort pe `array`, deși la nivel conceptual lista pare potrivită pentru interclasare. În teorie, lista permite relink-AREA nodurilor fără deplasarea masivă a elementelor.

În practică, însă, nodurile sunt dispersate în memorie, iar accesul pointer-cu-pointer produce localitate slabă. O cercetare viitoare ar trebui să măsoare direct cache miss-urile și accesările în memoria principală. În acest fel s-ar putea explica mai precis de ce avantajul teoretic al listei este depășit de costul real al memoriei.

A doua direcție privește comparația dintre Merge Sort paralel și QuickSort cu pivot random. Rezultatele actuale arată că paralelizarea Merge Sort nu a fost suficientă pentru a depăși QuickSort-ul în testele mari. O analiză viitoare ar trebui să separe timpul petrecut în sortarea efectivă, timpul de creare și sincronizare a thread-urilor, timpul de interclasare și traficul suplimentar în memorie. Această direcție este importantă deoarece ar putea arăta pragul de la care paralelizarea devine rentabilă și condițiile în care un algoritm paralel poate depăși o implementare secvențială foarte eficientă.

A treia direcție este analiza mecanismului de branch prediction. În această lucrare, explicațiile s-au concentrat mai ales pe complexitate, structură de date și localitate în memorie. Totuși, procesoarele moderne sunt afectate și de predicția ramificațiilor. În QuickSort, comparațiile din bucla de partiționare pot produce ramificații greu de prezis, iar acest lucru poate afecta pipeline-ul procesorului. Lucrările lui Kaligosi și Sanders [9], respectiv Edelkamp și Weiss [7], arată că branch misprediction-ul poate influența serios performanța QuickSort. O continuare firească ar fi măsurarea acestor efecte pe implementările testate.

A patra direcție este studierea mai atentă a algoritmilor hibridi moderni. În lucrarea de față a fost analizat IntroSort, dar există și alte abordări relevante. Timsort [14] este important pentru date aproape sortate, deoarece profită de secvențele deja ordonate existente în input. Pattern-defeating Quicksort [13] este relevant pentru cazurile în care datele au tipare ce pot afecta negativ un QuickSort obișnuit. O comparație viitoare între IntroSort, Timsort, PDQsort și implementările standard ar completa analiza realizată aici.

A cincea direcție este păstrarea și organizarea datelor experimentale pentru verificare și studiu ulterior. Datele obținute nu trebuie privite doar ca rezultate finale, ci ca material de cercetare. Ele permit refacerea graficelor, verificarea concluziilor, compararea implementărilor viitoare și observarea unor variații care nu sunt evidente la prima analiză. O bază experimentală bine păstrată ar face posibilă extinderea cercetării fără reluarea completă a tuturor testelor.

A șasea direcție este analiza posibilității de normalizare a tipurilor de date. Pentru valori reale, dacă se cunoaște numărul maxim de zecimale, acestea ar putea fi scalate cu 10^p și convertite la întregi. Această metodă ar putea uniformiza infrastructura de testare și ar reduce diferențele dintre comparațiile pe tipuri de date diferite. Totuși, problema trebuie tratată atent, deoarece poate apărea overflow, mai ales când valorile sunt mari sau precizia este ridicată. Pentru caractere, conversia la valori întregi este naturală, dar câștigul real de performanță trebuie măsurat experimental.

A șaptea direcție privește sortarea pe GPU. În această lucrare, placa video a fost menționată doar ca parte a configurației hardware, iar testele au fost executate pe procesor. Pentru volume foarte mari de date, literatura arată că sortarea pe arhitecturi many-core poate deveni importantă [15]. O continuare posibilă ar fi compararea algoritmilor CPU cu variante GPU, cum ar fi implementări de Radix Sort sau Bitonic Sort adaptate pentru plăci grafice moderne.

În ansamblu, aceste direcții pornesc din aceleași întrebări deschise de rezultate: cât contează memoria cache, cât contează branch prediction-ul, când merită paralelizarea, cum se comportă algoritmi hibridi moderni și cât de mult poate fi influențată performanța de reprezentare a datelor. Astfel, cercetarea poate continua de la o comparație generală a algoritmilor spre o analiză mai fină a arhitecturii hardware și a implementărilor reale.

11. Concluzii

Lucrarea a analizat experimental mai mulți algoritmi de sortare, urmărind diferența dintre complexitatea teoretică și performanța practică. Rezultatele confirmă faptul că algoritmi de complexitate $O(n^2)$, precum Bubble Sort, Selection Sort și Binary Insertion Sort, devin rapid

impracticabili pentru volume mari de date. Acești algoritmi rămân utili pentru înțelegerea mecanismelor de bază ale sortării, dar nu reprezintă soluții potrivite pentru inputuri mari.

Algoritmii de complexitate $O(n \log n)$ au avut rezultate mult mai bune. QuickSort a fost foarte eficient în testele mari, mai ales pe structuri contigue. Merge Sort a avut un comportament predictibil, dar a fost penalizat de memoria auxiliară și de operațiile de interclasare. IntroSort manual a evidențiat importanța algoritmilor hibridi, dar și faptul că performanța depinde foarte mult de calitatea implementării concrete.

Un rezultat important al lucrării este influența structurii de date. **Array**-ul a fost, în general, cea mai eficientă structură, datorită stocării contigue și localității bune în memorie. **Vector**-ul a avut rezultate apropiate, iar lista simplu înlănțuită a fost penalizată de accesul secvențial, dereferențierile repetate și localitatea slabă. Acest lucru arată că structura de stocare poate schimba semnificativ timpul real de execuție, chiar atunci când algoritmul rămâne același.

Rezultatele privind Merge Sort paralel arată că paralelizarea nu garantează automat o performanță mai bună. Deși varianta paralelă a redus timpul față de Merge Sort clasic, aceasta nu a depășit QuickSort-ul cu pivot random în testele mari. Costurile de sincronizare, interclasare și trafic suplimentar în memorie pot reduce avantajul teoretic al execuției concurente.

Comparația cu literatura de specialitate confirmă concluziile teoretice cunoscute, dar le completează prin date experimentale. Complexitatea asimptotică rămâne un criteriu esențial, însă nu este suficientă pentru a explica singură performanța reală. În practică, contează și structura de date, distribuția inputului, localitatea în memorie, predicția ramificațiilor, memoria auxiliară și detaliile implementării.

Contribuția lucrării constă în realizarea unei comparații experimentale între algoritmi de sortare cunoscuți, pe structuri de date diferite și pe distribuții variate ale inputului. Lucrarea nu propune un algoritm nou, ci evidențiază modul în care factori practici pot modifica performanța observată față de așteptările teoretice. Astfel, studiul arată că alegerea unui algoritm de sortare trebuie făcută prin combinarea teoriei cu măsurători practice.

În ansamblu, lucrarea susține ideea că analiza algoritmilor nu trebuie să se oprească la notația asimptotică. Pentru a înțelege comportamentul real al unei metode de sortare, trebuie analizate împreună algoritmul, structura de date, implementarea, distribuția inputului și arhitectura pe care rulează programul. Aceasta este concluzia principală a studiului: teoria oferă direcția, dar experimentul arată cât de bine funcționează efectiv soluția.

Bibliografie

- [1] Nicolas Auger, Vincent Jugé, Cyril Nicaud, and Carine Pivoteau. Merge strategies: From merge sort to timsort. *arXiv preprint arXiv:1505.04729*, 2015.
- [2] Michael Axtmann, Sascha Witt, Daniel Ferizovic, and Peter Sanders. In-place parallel super scalar samplesort. In *25th Annual European Symposium on Algorithms*, 2017.
- [3] Jon L. Bentley and M. Douglas McIlroy. Engineering a sort function. *Software: Practice and Experience*, 23(11):1249–1265, 1993.
- [4] Paul Biggar, David Gregg, and Ed Hennessy. Sorting in the presence of branch prediction and caches. Technical report, Trinity College Dublin, 2005.
- [5] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms*. MIT Press, 3 edition, 2009.
- [6] cppreference.com. std::sort. <https://en.cppreference.com/w/cpp/algorithm/sort>.
- [7] Stefan Edelkamp and Armin Weiss. Blockquicksort: How branch mispredictions don't affect quicksort. *ACM Journal of Experimental Algorithmics*, 22:1–29, 2016.
- [8] C. A. R. Hoare. Quicksort. *The Computer Journal*, 5(1):10–16, 1962.
- [9] Kanela Kaligosi and Peter Sanders. How branch mispredictions affect quicksort. In *Algorithms – ESA 2006*, pages 780–791. Springer, 2006.
- [10] Donald E. Knuth. *The Art of Computer Programming, Volume 3: Sorting and Searching*. Addison-Wesley, 2 edition, 1998.

- [11] Anthony LaMarca and Richard E. Ladner. The influence of caches on the performance of sorting. In *Proceedings of the Eighth Annual ACM-SIAM Symposium on Discrete Algorithms*, pages 370–379, 1997.
- [12] David R. Musser. Introspective sorting and selection algorithms. *Software: Practice and Experience*, 27(8):983–993, 1997.
- [13] Orson Peters. Pattern-defeating quicksort. <https://github.com/orlp/pdqsort>, 2021.
- [14] Tim Peters. Timsort. Python Developer Documentation, 2002.
- [15] Nadathur Satish, Mark Harris, and Michael Garland. Designing efficient sorting algorithms for manycore gpus. In *2009 IEEE International Symposium on Parallel & Distributed Processing*, pages 1–10. IEEE, 2009.
- [16] Robert Sedgewick and Kevin Wayne. *Algorithms*. Addison-Wesley, 4 edition, 2011.
- [17] Philippas Tsigas and Yi Zhang. A simple, fast parallel implementation of quicksort and its performance evaluation on sun enterprise 10000. In *Proceedings of the 11th Euromicro Conference on Parallel, Distributed and Network-Based Processing*, pages 372–381, 2003.
- [18] J. W. J. Williams. Algorithm 232: Heapsort. *Communications of the ACM*, 7(6):347–348, 1964.